

Faydam PDKS Mobil Uygulama — API, QR ve APK Devir Rehberi

Mobil uygulamanın API'ye bağlanması, QR ile devam kaydı oluşturması ve test APK'sı üretilmesi için uygulama rehberi.

Bu belge mobil uygulamayı geliştiren kişiye doğrudan gönderilebilir. Amaç; Flutter uygulamasının Faydam PDKS API'ye bağlanması, QR ile giriş/çıkış kaydı göndermesi, kaydın web kontrol panelinde görülmesi ve test APK'sı üretilmesidir.

1. API nedir ve nerede bulunur?

API, mobil uygulama ile veritabanı/web sistemi arasındaki güvenli iletişim kapısıdır. Flutter uygulaması PostgreSQL veritabanına veya web sayfalarına doğrudan bağlanmaz. Yalnızca HTTP/HTTPS üzerinden [FaydamPDKS.Api](#) projesine istek gönderir.

Bu çözümde üç ayrı parça vardır:

- [FaydamPDKS.Web](#): Yönetici web paneli.
- [FaydamPDKS.Api](#): Mobil uygulamanın bağlanacağı backend API.
- PostgreSQL: Web ve API'nin ortak kullandığı veritabanı.

API kodları backend deposunda şu klasördedir:

```
FaydamPDKS.Api/
```

Mobil uygulama deposunda API'nin kendisi bulunmaz. Mobil uygulamaya yalnızca API'nin adresi ve bu API'yi çağıran Dart kodu yazılır.

2. Mobil uygulamaya verilecek API adresi

API temel adresi çalıştırılan ortama göre değişir.

Android emülatör ile yerel test

```
http://10.0.2.2:5055/api/v1
```

Android emülatörde `localhost` emülatörün kendisini gösterir. Bilgisayardaki API için `10.0.2.2` kullanılır.

Gerçek telefon ve aynı Wi-Fi ile test

```
http://BILGISAYARIN-YEREL-IP-ADRESI:5055/api/v1
```

Örnek:

```
http://192.168.1.35:5055/api/v1
```

API bilgisayarda yalnızca `localhost` yerine ağdan dinleyecek şekilde başlatılmalıdır:

```
dotnet run --project FaydamPDKS.Api --urls http://0.0.0.0:5055
```

Windows Güvenlik Duvarı 5055 portu için izin isteyebilir. Bu yöntem yalnızca kontrollü yerel test içindir.

İnternet üzerinden gerçek APK testi

Önerilen yöntem HTTPS kullanan bir test sunucusudur:

```
https://test-api.faydam.com/api/v1
```

Alan adı örnektir. Gerçek adres backend yayımlandıktan sonra mobil geliştiriciye verilir. Release APK içinde `localhost`, `10.0.2.2` veya geliştiricinin kişisel bilgisayar IP'si bırakılmamalıdır.

3. Flutter projesinde yapılacak ilk düzenlemeler

Mobil depoda ana Flutter projesi kök klasördür. İçeride ayrıca `faydam_pdksp/` adlı ikinci bir örnek Flutter projesi bulunuyor. Tek proje seçilmeli; aşağıdaki işlemler kökteki `lib/`, `android/` ve `pubspec.yaml` üzerinde yapılmalıdır.

pubspec.yaml

`dependencies` bölümüne ekleyin:

```
dio: ^5.8.0
flutter_secure_storage: ^9.2.4
uuid: ^4.5.1
device_info_plus: ^11.3.3
```

Ardından:

```
flutter pub get
```

Paketlerin güncel ve proje Flutter sürümüyle uyumlu kararlı sürümleri tercih edilebilir.

lib/config/api_config.dart

Bu dosyayı oluşturun:

```
class ApiConfig {
  static const String baseUrl = String.fromEnvironment(
    'API_BASE_URL',
    defaultValue: 'http://10.0.2.2:5055/api/v1',
  );
}
```

API adresini kaynak koda sabitlemeden APK üretmek için:

```
flutter build apk --debug \
  --dart-define=API_BASE_URL=https://test-api.faydam.com/api/v1
```

Windows PowerShell'de komut tek satır kullanılabilir:

```
flutter build apk --debug --dart-define=API_BASE_URL=https://test-api.faydam.com/api/v1
```

4. Android izinleri

Dosya:

```
android/app/src/main/AndroidManifest.xml
```

Mevcut depoda bazı `uses-permission` satırları `<manifest>` etiketinin dışında. Dosyanın başlangıcı şu yapıda olmalıdır:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <uses-permission android:name="android.permission.INTERNET" />
  <uses-permission android:name="android.permission.CAMERA" />
  <uses-permission android:name="android.permission.USE_BIOMETRIC" />

  <application
    android:label="Faydam PDKS"
    android:name="${applicationName}"
    android:icon="@mipmap/ic_launcher">
    <!-- Mevcut activity içeriği burada kalır. -->
  </application>
</manifest>
```

Yerel HTTP testinde Android cleartext trafiği engellerse yalnızca debug geliştirme yapılandırmasında izin verilmelidir. Üretim ve gerçek test sunucusunda HTTPS kullanılmalıdır.

5. Token saklama ve API servisi

Dosya:

```
lib/services/api_service.dart
```

Depoda bu dosya şu anda boştur. [Dio](#), access token ve refresh token yönetimi burada bulunmalıdır. Tokenlar [AppSettings](#) içinde yalnızca bellekte veya düz metin dosyada tutulmamalı; [flutter_secure_storage](#) kullanılmalıdır.

Temel servis örneği:

```
import 'package:dio/dio.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';
import '../config/api_config.dart';

class ApiService {
  ApiService() {
    : dio = Dio(BaseOptions(
      baseUrl: ApiConfig.baseUrl,
      connectTimeout: const Duration(seconds: 15),
      receiveTimeout: const Duration(seconds: 15),
      headers: {'Content-Type': 'application/json'},
    ));

    final Dio dio;
    final FlutterSecureStorage storage = const FlutterSecureStorage();

    Future<void> attachToken() async {
      final token = await storage.read(key: 'access_token');
      if (token != null) {
        dio.options.headers['Authorization'] = 'Bearer $token';
      }
    }

    Future<Map<String, dynamic>> login(String email, String password) async {
      final response = await dio.post('/auth/login', data: {
        'email': email,
        'password': password,
        'deviceName': 'Android telefon',
      });

      final data = Map<String, dynamic>.from(response.data);
      await storage.write(key: 'access_token', value: data['accessToken']);
      await storage.write(key: 'refresh_token', value: data['refreshToken']);
      await attachToken();
      return data;
    }

    Future<Response<dynamic>> createAttendanceEvent({
      required String eventType,
      required int zoneId,
      required String deviceEventId,
    }) async {
      await attachToken();
      return dio.post('/attendance/events', data: {
        'eventType': eventType,
        'occurredAt': DateTime.now().toIso8601String(),
        'deviceEventId': deviceEventId,
        'zoneId': zoneId,
      });
    }

    Future<Map<String, dynamic>> getTodayAttendance() async {
      await attachToken();
      final response = await dio.get('/attendance/today');
      return Map<String, dynamic>.from(response.data);
    }
  }
}
```

Üretim sürümünde 401 geldiğinde [/auth/refresh](#) bir kez çağrılmalı, yeni tokenlar kaydedilmeli ve ilk istek bir kez tekrarlanmalıdır. Refresh de başarısızsa tokenlar silinip giriş ekranına dönmelidir.

6. Yazılmış API uçlarının tam listesi

Tüm yollar aşağıdaki temel adresin devamıdır:

{API_BASE_URL} = https://test-api.faydam.com/api/v1

Metot	Yol	Kullanım
POST	/auth/login	E-posta/parola ile giriş ve token alma
POST	/auth/refresh	Access token yenileme
POST	/auth/logout	Mobil oturumu kapatma
GET	/me	Kullanıcının profilini alma
PUT	/me	Telefon ve bildirim tercihlerini güncelleme
GET	/me/export	Kişisel verileri dışa aktarma
GET	/attendance/today	Bugünkü giriş, çıkış ve çalışma özeti
GET	/attendance?from=YYYY-MM-DD&to=YY-MM-DD	Tarih aralığı puantajı
POST	/attendance/events	Mobil giriş veya çıkış kaydı oluşturma
GET	/attendance-corrections	Puantaj düzeltme talepleri
POST	/attendance-corrections	Yeni düzeltme talebi
GET	/leave-requests	Kullanıcının izin talepleri
POST	/leave-requests	Yeni izin talebi
DELETE	/leave-requests/{id}	Bekleyen izin talebini iptal etme
GET	/notifications	Bildirimleri listeleme
POST	/notifications/{id}/read	Bildirimi okundu yapma

Login ve refresh dışındaki bütün çağrılarda şu başlık zorunludur:

Authorization: Bearer ACCESS_TOKEN

Giris

```
POST /api/v1/auth/login
Content-Type: application/json

{
  "email": "personel@faydam.com",
  "password": "123456",
  "deviceName": "Samsung A54"
}
```

Başarılı yanıtta [accessToken](#), [refreshToken](#), [expiresAt](#) ve [user](#) döner.

Token yenileme

```
POST /api/v1/auth/refresh
Content-Type: application/json

{
  "refreshToken": "LOGIN-YANITINDA-GELEN-TOKEN"
}
```

Buğünkü puantai

```
GET /api/v1/attendance/today
Authorization: Bearer ACCESS_TOKEN
```

Örnek yanıt:

```
{
  "workDate": "2026-07-16",
  "status": "Complete",
  "firstEntry": "2026-07-16T08:57:12+03:00",
  "lastExit": "2026-07-16T18:05:00+03:00",
  "workedMinutes": 488,
  "expectedMinutes": 480,
  "lateMinutes": 0,
  "overtimeMinutes": 8
}
```

İzin talebi

```
POST /api/v1/leave-requests
Authorization: Bearer ACCESS_TOKEN
Content-Type: application/json
```

```
{
  "leaveType": "Annual",
  "startDate": "2026-07-20",
  "endDate": "2026-07-24",
  "reason": "Yıllık izin"
}
```

leaveType: Annual, Sick, Excuse veya Unpaid.

Puantai düzeltme talebi

```
POST /api/v1/attendance-corrections
Authorization: Bearer ACCESS_TOKEN
Content-Type: application/json
```

```
{
  "workDate": "2026-07-15",
  "requestedEntry": "09:00:00",
  "requestedExit": "18:00:00",
  "reason": "Terminal çökme kaydımı oluşturmadım."
}
```

7. QR ile giriş/çıkış işlemi

Test QR içeriği

İlk uçtan uca testte iki ayrı QR kullanılabilir.

Giriş QR:

```
{"version":1,"eventType":"Entry","zoneId":1,"locationName":"Faydam Merkez"}
```

Çıkış QR:

```
{"version":1,"eventType":"Exit","zoneId":1,"locationName":"Faydam Merkez"}
```

zoneId değeri tahmin edilmemeli; web/backend veritabanındaki gerçek aktif bölge kimliği kullanılmalıdır.

lib/qrcode_screen.dart içinde yapılacak işlem

Mevcut _processQRCode yalnızca mesaj gösteriyor. Bunun yerine:

1. QR metnini `jsonDecode` ile çöz.
2. `eventType` değerinin `Entry` veya `Exit` olduğunu doğrula.
3. `zoneId` değerinin pozitif sayı olduğunu doğrula.

- Her okutma için yeni UUID üret.
- `ApiService.createAttendanceEvent` çağır.
- Yalnızca HTTP 201 döndüğünde başarılı mesajı göster.
- Ardından `/attendance/today` çağırısıyla kaydı tekrar doğrula.

Örnek çekirdek kod:

```
import 'dart:convert';
import 'package:uuid/uuid.dart';
import 'services/api_service.dart';

final api = ApiService();
const uuid = Uuid();

Future<void> processQr(String rawValue) async {
  final qr = Map<String, dynamic>.from(jsonDecode(rawValue));
  final eventType = qr['eventType'] as String?;
  final zoneId = qr['zoneId'] as int?;

  if (![ 'Entry', 'Exit' ].contains(eventType) || zoneId == null || zoneId < 1) {
    throw Exception('Geçersiz Faydam QR kodu.');
```

API aynı `deviceEventId` ikinci kez gönderildiğinde 409 `DUPLICATE_EVENT` döndürür. Bu, bağlantı tekrarlarında çift kayıt oluşmasını engeller.

Üretim güvenliği

Yukarıdaki düz JSON QR ilk entegrasyon testi içindir. Kullanıcı bu QR'ın fotoğrafını çekip başka yerde okutabilir. Gerçek kullanım öncesinde aşağıdakilerden biri uygulanmalıdır:

- Sunucu tarafından imzalanmış ve kısa süreli QR tokeni,
- Terminal tarafından sürekli yenilenen QR challenge,
- Konum/Bluetooth/NFC gibi ikinci doğrulama.

Mevcut API QR imzası doğrulamıyor. Bu nedenle ilk APK "entegrasyon test APK'sı" olarak değerlendirilmelidir; güvenli üretim APK'sı olarak dağıtılmamalıdır.

8. Login ekranında değiştirilecek yer

Dosya:

```
lib/login_screen.dart
```

Mevcut sabit kontrol kaldırılmalıdır:

```
if (email == "yonetici2@faydam.com" && password == "12345678")
```

Yerine:

```
try {
  final result = await ApiService().login(email, password);
  final user = Map<String, dynamic>.from(result['user']);
```

```

if (!mounted) return;
context.read<AppSettings>().loginSuccess(
  result['accessToken'],
  user['id'],
  user['fullName'],
  user['email'],
);

Navigator.pushReplacement(
  context,
  MaterialPageRoute(builder: (_) => const MainScreen()),
);
} on DioException catch (error) {
  final message = error.response?.data?['message'] ?? 'API bağlantısı kurulamadı.';
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text(message.toString()))),
  );
}

```

9. Kaydın web paneline düşmesi

POST /attendance/events başarılı olduğunda API:

- Oturumdaki personelin kimliğini JWT'den alır.
- Kaydı ortak [AccessLogs](#) tablosuna yazar.
- [LogType](#) değerini [Giris](#) veya [Cikis](#) yapar.
- [Source](#) değerini [Mobile](#) yapar.
- Bölgeyi [zoneId](#) üzerinden ilişkilendirir.

Web kontrol panelindeki “Son hareketler” aynı [AccessLogs](#) tablosunu okur. Bu nedenle başarılı mobil olay, web sayfası yenilendiğinde listede görünür.

10. Uçtan uca test sırası

1. PostgreSQL, API ve web aynı veritabanıyla çalıştırılır.
2. Web panelinden test personeli oluşturulur.
3. Personele geçerli işyeri/bölüm ve parola atanır.
4. Veritabanında veya backend üzerinden aktif [zoneId](#) öğrenilir.
5. Bu [zoneId](#) ile giriş ve çıkış QR'ları hazırlanır.
6. Flutter APK doğru [API_BASE_URL](#) ile üretilir.
7. APK telefona kurulur.
8. Test personeli mobilde oturum açar.
9. Giriş QR'ı okutulur; mobilde HTTP 201 alınır.
10. /attendance/today yanıtında [firstEntry](#) dolu görülür.
11. Web kontrol paneli yenilenir; “Giriş” hareketi görülür.
12. Çıkış QR'ı okutulur; mobilde HTTP 201 alınır.
13. /attendance/today yanıtında [lastExit](#) dolu görülür.
14. Web paneli yenilenir; “Çıkış” hareketi görülür.
15. Aynı [deviceEventId](#) tekrar gönderilir; HTTP 409 beklenir.
16. Yanlış/bozuk tokenla istek gönderilir; HTTP 401 beklenir.

11. APK üretme

Mobil geliştiricinin bilgisayarında Flutter ve Android SDK kurulu olmalıdır.

Kontroller:

```
flutter doctor
flutter pub get
flutter analyze
flutter test
```

Debug APK:

```
flutter build apk --debug --dart-define=API_BASE_URL=https://test-api.faydam.com/api/v1
```

Çıktı:

```
build/app/outputs/flutter-apk/app-debug.apk
```

Release APK ancak testler tamamlandıktan ve Android imzalama anahtarı güvenli biçimde hazırlandıktan sonra üretilmelidir:

```
flutter build apk --release --dart-define=API_BASE_URL=https://test-api.faydam.com/api/v1
```

Release keystore dosyası, parolası, JWT anahtarı veya veritabanı parolası GitHub'a yüklenmemelidir.

12. Arkadaşınıza gönderilecekler

- Bu doküman.
- Kesin test API adresi.
- Swagger adresi: <https://TEST-API-ADRESI/swagger>.
- Test personeli e-posta adresi ve geçici parolası (güvenli kanaldan).
- Giriş ve çıkış QR görselleri.
- Kullanılacak gerçek [zoneId](#).
- Beklenen test senaryosu ve web paneli adresi.

API adresi ve gerçek [zoneId](#) belirlenmeden APK'daki QR akışı uçtan uca doğrulanamaz.